Plano de Ação para Classificação dos Testes (Smoke, Regression e E2E)

Objetivo

Classificar os cenários existentes em categorias de execução que permitam uma estratégia de testes eficiente durante o ciclo de desenvolvimento:

- **Smoke:** validações rápidas para verificar se o sistema está funcional após integração de código.
- **Regression:** validação ampla das funcionalidades entregues.
- **End-to-End (E2E):** validação completa de jornadas críticas do usuário.

Como o agente de IA não possui acesso ao código, o plano foi construído para que a classificação seja realizada com base apenas nos cenários de negócio existentes.

Etapa 1 - Definir Critérios de Classificação

Smoke

Classifique como Smoke os cenários que:

- Validam funcionalidades críticas do sistema.
- Possuem alta probabilidade de detectar falhas graves.
- Executam rapidamente.
- Não dependem de fluxos extensos.
- Representam a saúde geral da aplicação.

Critérios:

✅ Login funciona

✅ Navegação principal funciona

✅ Processo principal de compra funciona

✅ Carrinho funciona

❌ Casos de erro detalhados

❌ Validações de borda

❌ Cenários secundários

Regression

Classifique como Regression os cenários que:

- Validam regras funcionais completas.
- Exercitam comportamentos positivos e negativos.
- Verificam funcionalidades recentemente alteradas.
- Garantem que funcionalidades antigas não quebraram.

Critérios:

✅ Todos os cenários funcionais

✅ Casos positivos

✅ Casos negativos

✅ Regras de negócio

✅ Tratamento de erro

End-to-End (E2E)

Classifique como E2E os cenários que:

- Simulam a jornada completa do usuário.
- Exercitam múltiplas funcionalidades encadeadas.
- Validam integrações entre módulos.

Critérios:

✅ Login → Compra → Carrinho → Checkout → Logout

✅ Fluxos completos de negócio

✅ Cenários próximos ao uso real do usuário

❌ Cenários isolados

Etapa 2 - Analisar Cada Cenário

Funcionalidade: Login de usuário

Cenário: Login com sucesso

Classificação:

- Smoke ✅
- Regression ✅
- E2E ✅

Justificativa:

É pré-requisito para praticamente toda operação autenticada.

Cenário: Login com credenciais inválidas

Classificação:

- Regression ✅

Justificativa:

Valida regra de negócio e tratamento de erro.

Não é crítico para Smoke.

Não representa jornada E2E.

Cenário: Usuário bloqueado

Classificação:

- Regression ✅

Justificativa:

Validação de regra específica de autenticação.

Funcionalidade: Processo de Compra

Cenário: Compra de produto com sucesso

Classificação:

- Smoke ✅
- Regression ✅
- E2E ✅

Justificativa:

Fluxo principal de geração de valor do produto.

Se falhar, o sistema está comprometido.

Funcionalidade: Carrinho de Compras

Cenário: Adicionar item ao carrinho

Classificação:

- Smoke ✅
- Regression ✅
- E2E ✅

Justificativa:

Etapa obrigatória da jornada de compra.

Cenário: Remover item do carrinho

Classificação:

- Regression ✅

Justificativa:

Funcionalidade importante, porém não essencial para Smoke.

Funcionalidade: Ordenação de Produtos

Cenário: Ordenar produtos por preço crescente

Classificação:

- Regression ✅

Justificativa:

Melhoria de usabilidade.

Não impacta diretamente o negócio principal.

Cenário: Ordenar produtos por nome

Classificação:

- Regression ✅

Justificativa:

Mesmo racional da ordenação por preço.

Funcionalidade: Continuação de Compras

Cenário: Continuar comprando após adicionar item

Classificação:

- Regression ✅

Justificativa:

Fluxo complementar.

Funcionalidade: Validação do Checkout

Cenário: Checkout sem preencher dados

Classificação:

- Regression ✅

Justificativa:

Validação negativa do formulário.

Cenário: Checkout parcial

Classificação:

- Regression ✅

Justificativa:

Regra de validação do checkout.

Funcionalidade: Logout

Cenário: Logout com sucesso

Classificação:

- Regression ✅
- E2E ✅

Justificativa:

Normalmente representa o encerramento correto da sessão do usuário.

Funcionalidade: Resetar Estado da Aplicação

Cenário: Resetar carrinho

Classificação:

- Regression ✅

Justificativa:

Operação administrativa/auxiliar.

Baixa criticidade para Smoke.

Funcionalidade: Carrinho Vazio

Cenário: Acessar carrinho sem itens

Classificação:

- Regression ✅

Justificativa:

Validação de comportamento alternativo.

Etapa 3 - Consolidar Matriz de Classificação

Cenário	Smoke	Regression	E2E
Login com sucesso	✅	✅	✅
Login com credenciais inválidas	❌	✅	❌
Usuário bloqueado	❌	✅	❌
Compra de produto com sucesso	✅	✅	✅
Adicionar item ao carrinho	✅	✅	✅
Remover item do carrinho	❌	✅	❌
Ordenar produtos por preço crescente	❌	✅	❌
Ordenar produtos por nome	❌	✅	❌
Continuar comprando após adicionar item	❌	✅	❌
Checkout sem preencher dados	❌	✅	❌
Checkout parcial	❌	✅	❌
Logout com sucesso	❌	✅	✅
Resetar carrinho	❌	✅	❌
Acessar carrinho sem itens	❌	✅	❌

Etapa 4 - Definir Tags para Behave

Orientar o agente a adicionar as seguintes tags:

```
@smoke
@login
Cenário: Login com sucesso
```

```
@regression
@login
Cenário: Login com credenciais inválidas
```

```
@regression
@login
Cenário: Usuário bloqueado
```

```
@smoke
@regression
@e2e
@purchase
Cenário: Compra de produto com sucesso
```

```
@smoke
@regression
```

```
3  @e2e
4  @cart
5  Cenário: Adicionar item ao carrinho
```

E seguir a mesma estratégia para os demais cenários.

Etapa 5 - Configurar Estratégia de Execução

Pipeline CI (Smoke)

Executar a cada Pull Request:

```
1  behave --tags=@smoke
```

Objetivo:

- Verificar integração contínua.
- Detectar falhas rapidamente.
- Feedback rápido ao desenvolvedor.

Pipeline de Regressão

Executar:

- Merge para branch principal.
- Deploy em homologação.

```
1  behave --tags=@regression
```

Objetivo:

- Garantir estabilidade funcional.

Pipeline E2E

Executar:

- Antes da publicação em produção.
- Execução agendada diária (opcional).

```
1  behave --tags=@e2e
```

Objetivo:

- Garantir que as jornadas críticas continuem funcionando.

Tarefa pronta para enviar ao Agente de IA

```
Analise os cenários BDD existentes e aplique a seguinte classificação:

SMOKE
- Login com sucesso
- Adicionar item ao carrinho
- Compra de produto com sucesso

REGRESSION
- Todos os cenários do projeto

 E2E
 - Login com sucesso
 - Adicionar item ao carrinho
 - Compra de produto com sucesso
 - Logout com sucesso

 Para cada cenário:
 1. Adicione as tags apropriadas.
 2. Mantenha compatibilidade com Behave.
 3. Não altere o texto dos cenários.
 4. Gere uma tabela resumo contendo:
    Cenário | Smoke | Regression | E2E
 5. Considere que os testes serão executados via:
    - behave --tags=@smoke
    - behave --tags=@regression
    - behave --tags=@e2e
```

Sumário

A classificação foi construída com base em uma estratégia amplamente utilizada em pipelines de QA:

- **Smoke:** apenas os fluxos mais críticos para detectar rapidamente falhas causadas por integrações.
- **Regression:** cobertura completa das funcionalidades e regras de negócio.
- **E2E:** jornadas completas que representam o uso real do sistema pelo usuário final.

O resultado busca equilibrar velocidade de execução, cobertura funcional e confiabilidade das entregas.

Boas Práticas

1. Evite colocar mais de 10 a 15 cenários na suíte Smoke.
2. Mantenha a suíte Smoke com tempo de execução inferior a 10 minutos.
3. Utilize múltiplas tags por cenário (@smoke @regression) para evitar duplicação.
4. Revise a classificação a cada Sprint para refletir novas funcionalidades críticas.
5. Crie uma convenção de nomenclatura de tags:

```
@smoke
@regression
@e2e
@login
@cart
@checkout
@purchase
```

1. No GitHub Actions, crie jobs independentes para cada suíte, permitindo execução paralela.
2. Monitore continuamente quais testes falham com mais frequência; cenários instáveis não devem permanecer na suíte Smoke.
3. Para o Guará Framework, centralize a definição das tags na camada BDD (Behave), mantendo Page Transactions independentes da estratégia de execução.